

Setu

A Local-First Cross-Border Wealth and Portfolio Agent

Final Capstone Project Report | August 29, 2026

github.com/gadodia/Setu

1. Project title

Setu: A Local-First Cross-Border Wealth and Portfolio Agent

2. Problem and user

Setu is designed for an individual who holds financial assets in both the United States and India. That person may have brokerage and retirement accounts in the US, mutual funds and bank balances in India, and insurance products such as LIC endowment plans, ULIPs, or term policies. These records use different currencies, layouts, labels, and valuation rules. A normal portfolio tracker may show only market investments and miss policy obligations, source quality, or the difference between insurance coverage and current wealth.

The practical problem is therefore larger than listing assets. The user needs one evidence-backed view of net worth, cost and return, allocation, currency exposure, liquidity, insurance commitments, and important data gaps. They also need to know which source produced each value and retain control over whether that source participates in the portfolio.

3. System goal and scope

Setu's goal is to turn supported US and Indian financial PDFs into a normalized personal ledger and then explain the portfolio's current condition. It supports investment statements with optional cost basis, bank balances, and LIC, endowment, ULIP, and term-policy documents. It calculates base-currency net worth, allocation, document-backed ROI, concentration, currency exposure, known premium obligations, and a five-part portfolio health diagnostic. A user can pause on uncertain records, accept or reject a review, and deactivate a document so all linked data is excluded from calculations.

The system is deliberately read-and-advise only. It does not trade, move money, submit forms, predict markets, or provide personalized tax advice. It also does not guess missing insurance surrender values or treat a future maturity projection as present wealth. These limits keep the capstone focused on grounded portfolio understanding rather than unsafe financial action.

4. Final system architecture

The architecture follows one governing rule: **models interpret, deterministic code verifies and calculates, and humans resolve uncertainty.**

The first path is a LangGraph document-ingestion workflow. The orchestrator starts a graph run and passes the PDF to the ingestion role. Embedded text is read with pdfplumber; optional local OCR is used only when a page does not contain enough usable text. Bank balances use a deterministic parser. Holdings and policy documents can use qwen2.5:7b through local Ollama to map irregular text into typed Pydantic records. This keeps raw financial document text on the local machine.

The reconciliation role independently chooses the most authoritative stated total and compares it with the extracted records using Decimal arithmetic and a configurable 0.5% tolerance. LangGraph routes a valid result

to idempotent persistence, retries one mismatch, or pauses at a human-review interrupt. Accepted records enter a SQLAlchemy/SQLite ledger with source and date provenance; rejected records do not change financial truth. A separate SQLite checkpointer preserves in-progress workflow state by thread ID without storing raw policy text.

The second path is optional portfolio Q&A. Claude cannot read PDFs or write to the ledger. It can choose from three bounded, read-only tools: FX conversion, net-worth calculation, and portfolio analysis. Python and SQL execute those tools, and Claude explains the returned JSON. The loop is limited to four tool rounds and 1,200 output tokens. Components exchange typed state or structured tool results rather than free-form agent conversations.

This is a role-separated multi-agent system implemented as a controlled graph: coordinator, ingestion, reconciliation, deterministic financial analysis, and explanation. Setu does not use CrewAI. Research and form-filling agents remain future, separately permissioned workflows.

5. Design evolution across the program

The first concept was a broad personal-finance assistant covering transactions, reminders, forecasting, research, and wealth. Real implementation failures made the final system narrower and stronger. The capstone now concentrates on exact consolidation, evidence quality, and safe portfolio diagnostics. This created measurable outcomes instead of a long feature list.

The financial spine also moved away from model arithmetic. Ollama interprets documents and Claude explains approved results, but Python Decimal and SQL calculate every stored or displayed figure. Testing bank certificates and insurance documents showed that one generic holdings extractor was not enough, so ingestion was split into document-specific paths that still converge on one ledger.

Checkpoint 3 proposed semantic retrieval for policy rules, past runs, and user corrections. The implemented safety-critical rule set was small, and retrieving a plausible rule for the wrong policy could be worse than retrieving nothing. The final MVP therefore uses versioned deterministic policy semantics and exact SQL queries. Retrieval is deferred to future public reference documents and user notes, where sources can be cited and cannot override a failed valuation check.

The planned full Tree-of-Thought and generator/critic design was similarly reduced. Reconciliation now evaluates a bounded set of independent total hypotheses, ranks them by source authority, retries once, and escalates unresolved evidence. This preserves structured exploration while remaining reproducible and easy to test.

6. Implementation overview

Setu is written in Python. LangGraph provides explicit workflow nodes, conditional routing, checkpointing, and human interruption. FastAPI serves a password-protected dashboard and upload API. SQLAlchemy and SQLite hold institutions, accounts, dated statements, holdings, balances, policies, policy values, obligations, and source status. Pydantic validates model output before it reaches the ledger. Ollama provides optional local structured extraction, while the Anthropic API provides the separate explanatory tool loop.

Source-aware queries select only the latest active statement for each account. SHA-256 file hashes, statement-period constraints, and idempotent persistence prevent repeated uploads from inflating net worth. Insurance value is modeled separately from coverage: term coverage is never an asset; endowment value requires a stated current surrender value; and ULIP value requires a stated current fund value. The dashboard exposes data-source controls, policy details, review decisions, ROI, allocation, health components, and clear missing-data messages.

A one-command synthetic demo creates an isolated database, and a repeatable evaluation command checks the main correctness and safety claims without requiring personal data.

7. Evaluation and results

Evaluation uses a reproducible corpus of six synthetic documents and seven investment positions. The scenario includes US brokerage and retirement holdings, Indian mutual funds and cash, a valued ULIP, a term policy, and an LIC policy whose current surrender value is intentionally missing.

The final live-model run classified 6/6 documents correctly, passed 4/4 golden reconciliations, extracted 7/7 holdings with cost basis, triggered human review for a planted mismatch, reproduced net worth with zero error, and produced current, short-term, and long-term risk results. It exposed no raw document tool to Claude. The regression suite passed 110 tests with no failures.

The demo shows observable value beyond an inventory. It calculates \$275,488.50 net worth and 43.21% source-backed investment ROI, identifies 85.0% equity exposure, a 30.3% single-stock concentration, 87.1% USD exposure, and cash covering only 76.2% of known annual premiums. The explainable health result is 37/100, "Needs attention." It keeps the unsupported LIC current value unknown and excludes term coverage from wealth.

8. Safety and reliability

Setu combines preventive, detective, and human controls. The dashboard accepts only PDFs up to 25 MB, removes temporary uploads, requires a local password, and binds to 127.0.0.1. Raw documents stay on the local ingestion path. The project-local API key is ignored by Git, likely identifiers in typed questions are masked, and Claude has only sanitized read-only portfolio tools.

Typed schemas, exact arithmetic, source provenance, duplicate checks, independent reconciliation, and fail-closed insurance rules protect the ledger. Unknown currency, unusable extraction, conflicting policy evidence, and unresolved numeric mismatches stop or enter review instead of being silently accepted. Users can deactivate a source reversibly, which updates all linked calculations without deleting the audit trail. Setu has no trade, payment, or form-submission tool.

This design trades some speed and autonomy for reliability. Local models can be less flexible than a cloud model, deterministic rules require maintenance, and human review adds friction. For personal financial data, visible uncertainty and reproducible calculations are more important than automatic completion.

9. Limitations and next steps

The evaluation corpus is synthetic and too small to establish accuracy across every institution or PDF layout. OCR is optional and not yet measured on a broad scanned-document benchmark. FX rates are configured snapshots rather than live dated feeds. The health score is a transparent heuristic, not an investment-suitability assessment, and Setu lacks complete goals, liabilities, taxes, dependents, income stability, and emergency-expense inputs.

Next steps are to expand the anonymized golden corpus, add dated FX and market-price sources, add editable goals and liabilities, and package local model/OCR health checks. Retrieval can later support cited public product documents and user notes. Research and form-filling should be added only as separate workflows with their own permissions and approval gates.

10. Public GitHub repository

Repository: <https://github.com/gadodia/Setu>

The repository includes setup instructions, synthetic demo generation, evaluation commands, architecture and design-evolution documents, safety boundaries, tests, and a guided demo script. A reviewer can run `uv sync`, copy `.env.example`, set a dashboard password, and start the isolated demo with `uv run setu demo --reset` without a cloud API key or personal financial document.